CursoLab

Plataforma de cursos online, do catálogo ao certificado

Aluno Caio Bento
Professor Dener Santos

plataforma-cursos-omega.vercel.app

Nova Friburgo, 07/10/2026

Roteiro da apresentação

1. Contexto e solução
2. Processo e requisitos
3. Arquitetura geral
4. Front-end
5. Back-end e API
6. Controle de acesso
7. Modelo de dados e banco
8. Autenticação
9. Arquivos e implantação
10. Regras e desafios de integração
11. Testes e manutenção
12. Demonstração e conclusão

Contexto e solução

O problema

Uma plataforma de cursos precisa controlar quem pode assistir e até quando, em que ordem o aluno avança, como ele prova que aprendeu e como o instrutor acompanha tudo. Sem isso: conteúdo sem controle, gabarito exposto e certificado sem critério.

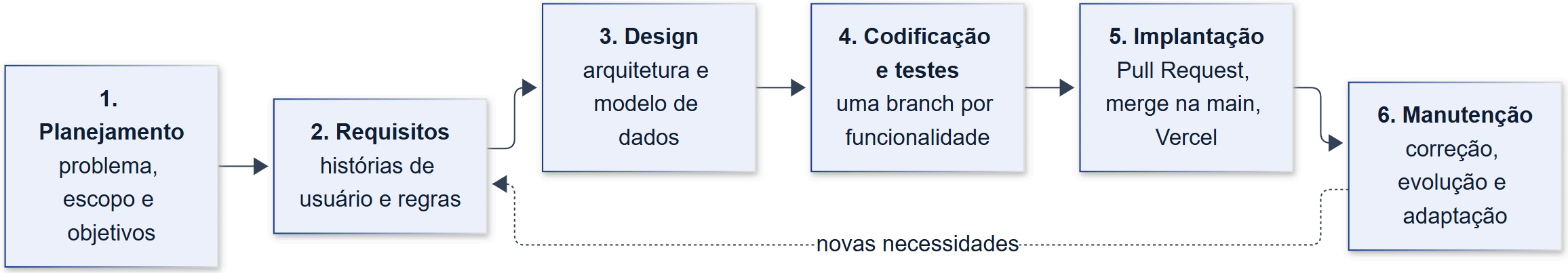
O que o CursoLab entrega

- área pública com landing e catálogo;
- área do aluno: compra, player, quiz e certificado;
- backoffice do instrutor: conteúdo, dúvidas e chamados;
- 6 cursos com conteúdo real: vídeos, quizzes e apostilas.

Princípio que guiou a arquitetura

- toda regra importante é decidida no servidor;
- a tela só apresenta o resultado;
- dados e regras protegidos também no banco;
- tudo publicado e testado contra a produção.

Processo de desenvolvimento



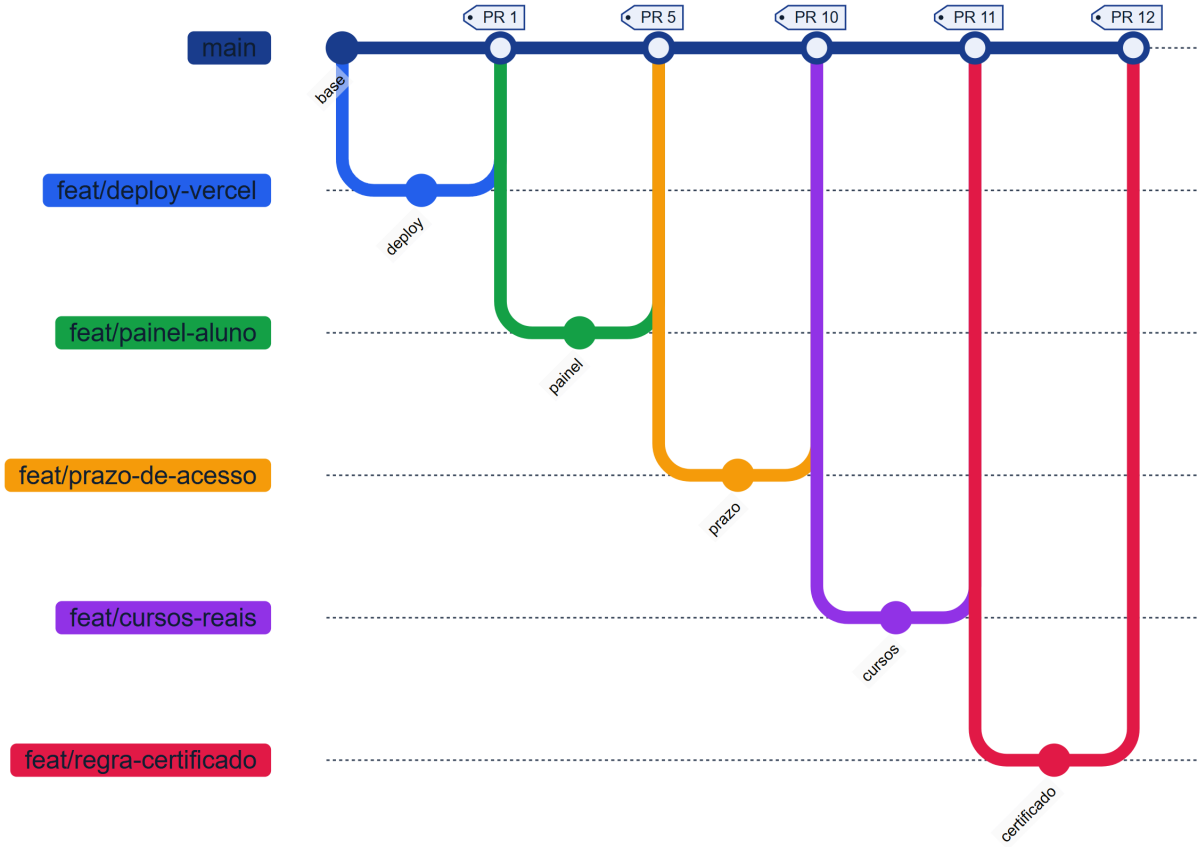
Ciclo adotado

- planejamento e escopo;
- requisitos em histórias de usuário;
- design da arquitetura e do banco;
- codificação e testes por funcionalidade;
- implantação automática;
- manutenção e novas necessidades.

Caracterização

Processo iterativo e incremental, com práticas ágeis, mas sem Scrum formal: sou um desenvolvedor só, sem sprints nem papéis definidos. Cada ciclo entregava uma funcionalidade completa e testada.

Processo na prática: fluxo de Git



Como cada mudança chegou ao ar

- uma branch por funcionalidade (feat/...);
- Pull Request com descrição e checklist de testes;
- merge na main publica sozinho na Vercel;
- mudanças no banco em migrações versionadas.

12
Pull Requests

37
commits

15
migrações SQL

Engenharia de requisitos

Requisitos funcionais (o que faz)

- cadastro, login e recuperação de senha;
- catálogo com busca e filtros;
- carrinho e checkout;
- player com progresso e quiz;
- certificado em PDF;
- dúvidas por aula e chamados;
- backoffice com editor e indicadores.

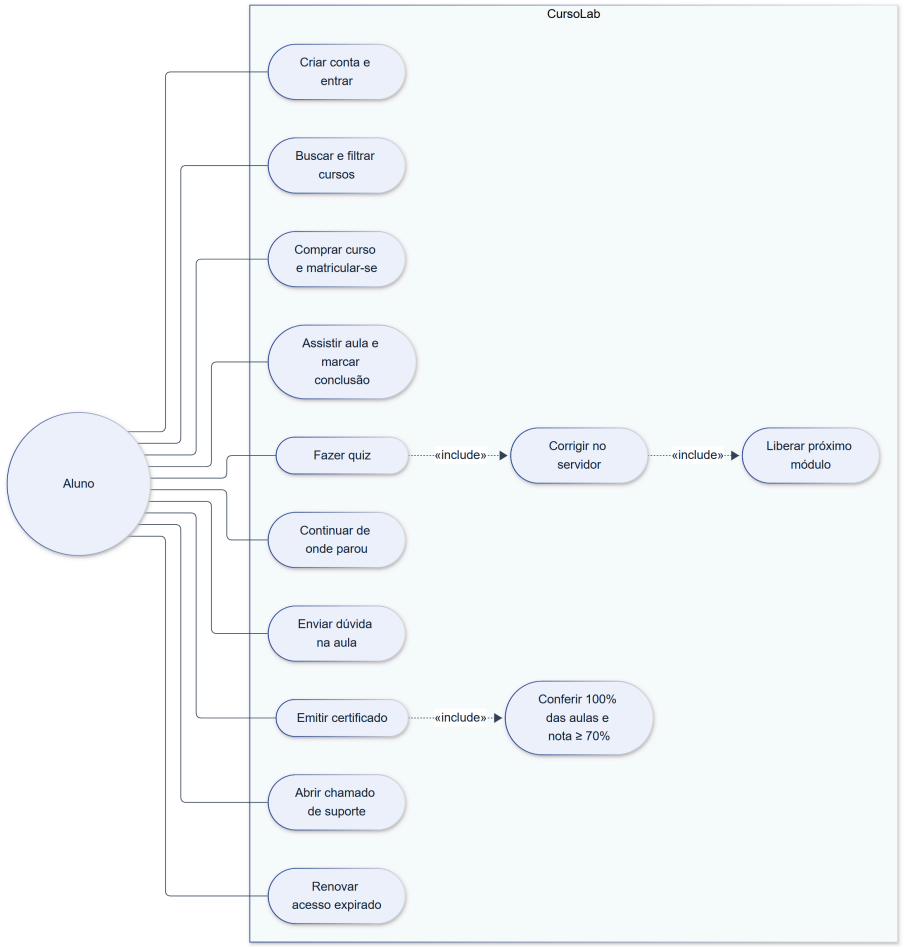
Requisitos não funcionais (como faz)

- segurança: senha com hash e tokens;
- chave secreta só no servidor;
- consistência: transação no banco;
- desempenho: upload direto ao Storage;
- usabilidade: linguagem visual única;
- disponibilidade: hospedado na nuvem.

Requisito volátil na prática

O certificado era liberado com 60% das aulas. A regra mudou para 100% das aulas e nota final ≥ 70%. Mudaram juntos: migração do banco, API, telas, testes e documentação.

Casos de uso



Aluno

- comprar, estudar, fazer quiz e continuar de onde parou;
- emitir certificado, tirar dúvidas e abrir chamados.

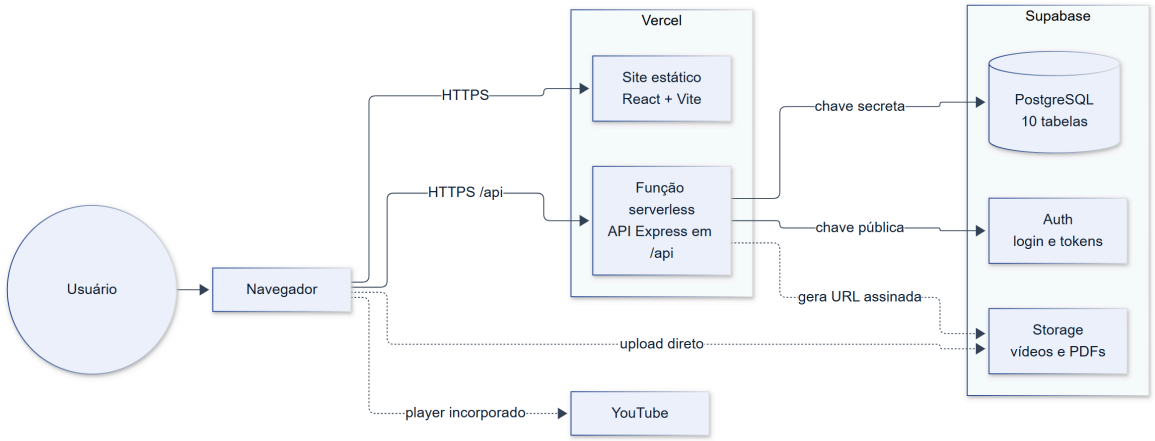
Administrador (instrutor)

- criar cursos e montar módulos, aulas e quizzes;
- enviar vídeos e materiais;
- responder dúvidas e chamados e ver indicadores.

Relações «include»

Fazer quiz inclui a correção no servidor, que libera o próximo módulo. Emitir certificado inclui conferir a regra.

Arquitetura geral: visão externa



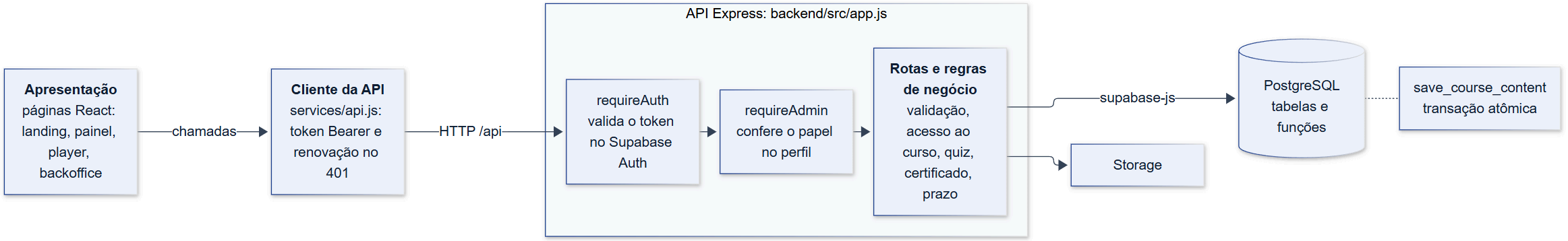
Visão de implantação

- o navegador baixa o site estático (React) da Vercel;
- chamadas em /api vão para uma função serverless com o Express;
- o Express fala com o Supabase: Auth, PostgreSQL e Storage;
- arquivos grandes vão direto do navegador ao Storage.

Decisão arquitetural

Monólito modularizado (uma API, um banco) servido como função serverless: simples de implantar e manter, sem servidor para administrar.

Arquitetura geral: camadas de uma requisição



Responsabilidades

- React: apresentação e navegação;
- api.js: cliente HTTP, token e renovação;
- requireAuth e requireAdmin: quem pode entrar;
- rotas: validação e regras de negócio;
- PostgreSQL: dados, restrições e transações.

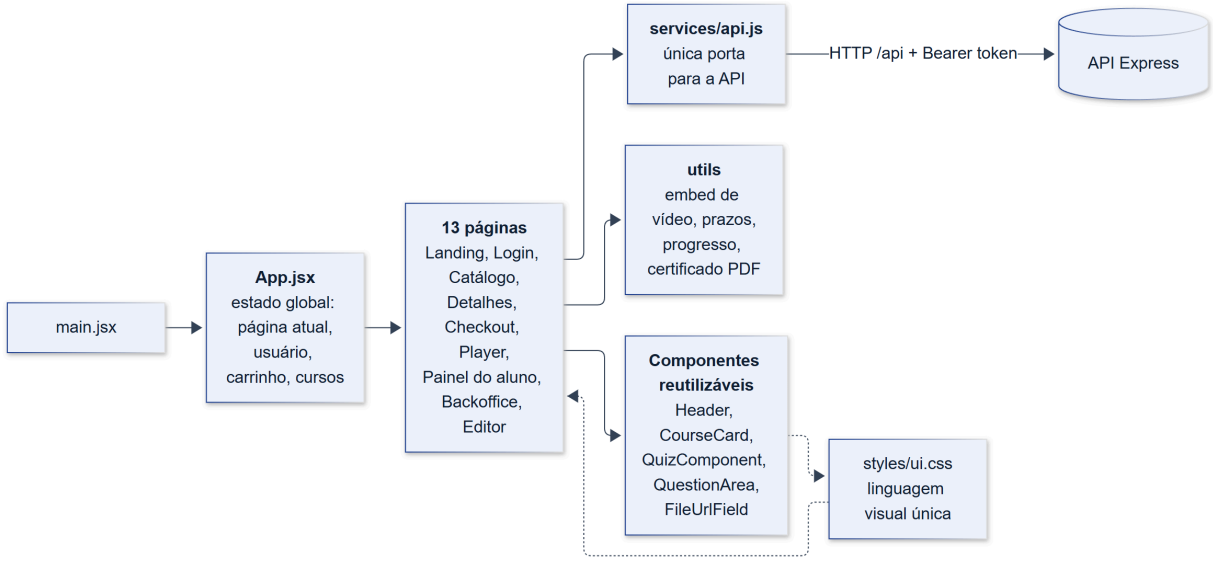
Vantagem

Cada camada tem uma responsabilidade clara. A tela não conhece o banco, e a regra de negócio não depende da tela.

Tecnologias principais

Tecnologia	Uso no projeto
React 19 + Vite	interface de página única e build do front-end
Node.js + Express 5	API REST com 36 rotas e middlewares de acesso
Supabase PostgreSQL	banco relacional: 10 tabelas, restrições e funções
Supabase Auth	contas, hash de senha e tokens JWT
Supabase Storage	vídeos, materiais e apostilas em PDF
jsPDF	certificado e apostilas gerados em PDF
Vercel	hospedagem, CDN, função serverless e deploy automático
GitHub	versionamento, branches e Pull Requests
ESLint + testes de integração	análise estática e 158 verificações automatizadas

Front-end: React como aplicação de página única



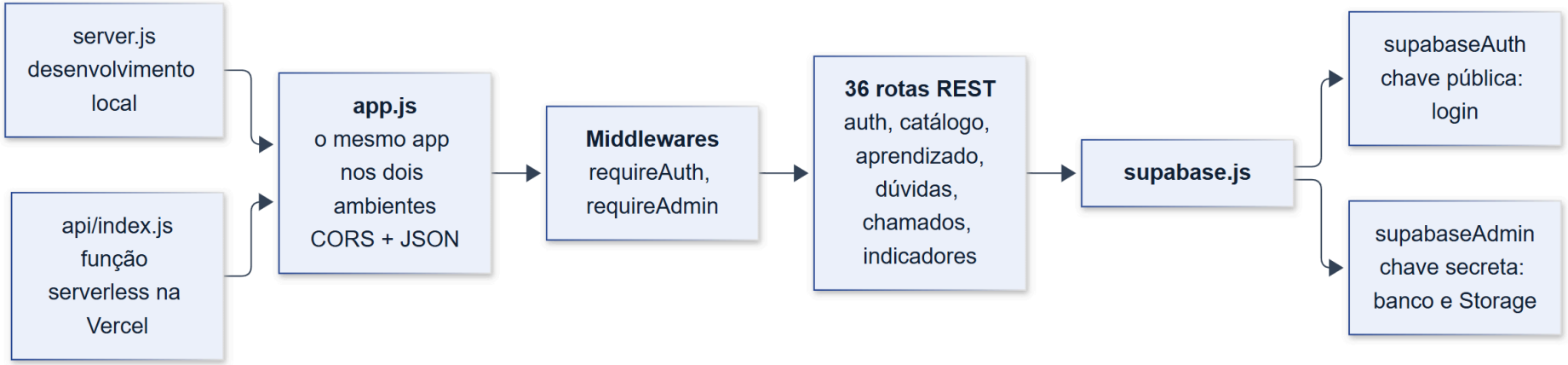
Como está organizado

- App.jsx guarda o estado global e decide qual página mostrar;
- 13 páginas montadas com componentes reutilizáveis;
- services/api.js é a única porta para o back-end;
- uma folha de estilos define a linguagem visual de tudo;
- o build do Vite gera arquivos estáticos para a CDN.

Decisão

O front-end não contém regra de negócio: ele chama a API e mostra a resposta. Isso permite mudar uma regra sem mexer nas telas e testar a API sem a interface.

Back-end: API Express em dois ambientes



Como está organizado

- app.js monta a API: CORS, JSON, middlewares e rotas;
- server.js roda o app localmente, na porta 3001;
- api/index.js reaproveita o mesmo app como função serverless;
- supabase.js cria os dois clientes: chave pública e secreta.

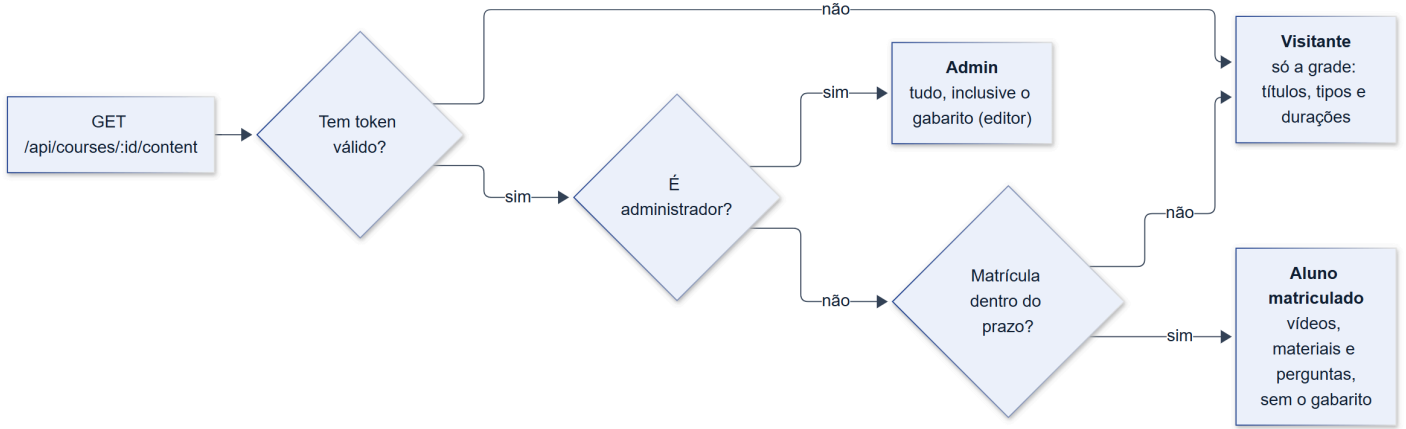
Por que isso importa

O código testado localmente é exatamente o que roda em produção. Só muda o ponto de entrada.

API REST: 36 rotas

Área	Rotas	Exemplos	Acesso
Autenticação e conta	9	POST /api/auth/login · /auth/refresh · /auth/password-recovery	público e aluno
Catálogo e conteúdo	9	GET /api/courses · PUT /courses/:id/content · POST /courses/:id/uploads	público e admin
Aprendizado	7	POST /api/enrollments · /lessons/:id/quiz-attempt · GET /enrollments/:id/certificate	aluno
Dúvidas	4	POST /api/lessons/:id/questions · PATCH /questions/:id/answer	aluno e admin
Chamados	5	POST /api/support-tickets · PUT /admin/support-tickets/:id	aluno e admin
Indicadores e saúde	2	GET /api/admin/analytics · GET /api/health	admin e público
Total	36	8 públicas · 16 exigem login · 12 só para administrador	

Controle de acesso: a mesma rota, três respostas



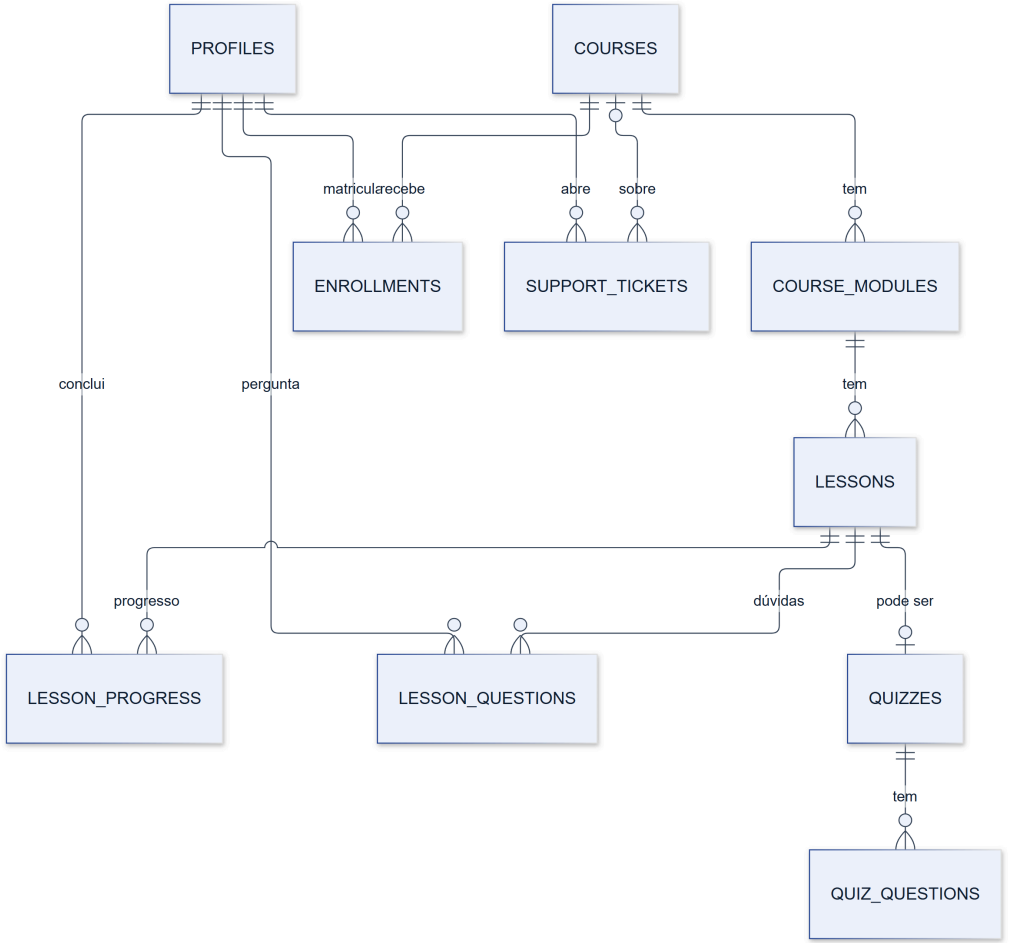
GET /api/courses/:id/content

- visitante: só a grade (títulos, tipos e durações);
- aluno sem matrícula ou com prazo vencido: só a grade;
- aluno matriculado: vídeos, materiais e perguntas, sem gabarito;
- administrador: tudo, inclusive o gabarito, para o editor.

Por que no servidor

Se a API mandasse tudo e a tela escondesse, bastaria abrir as ferramentas do navegador para ver o vídeo pago ou o gabarito.

Modelo de dados



10 tabelas em 4 grupos

- pessoas: profiles (aluno ou admin);
- catálogo: courses → course_modules → lessons → quizzes → quiz_questions;
- aprendizado: enrollments e lesson_progress;
- comunicação: lesson_questions e support_tickets.

Ponto central

A matrícula liga aluno e curso e guarda o prazo de acesso e a última aula aberta. O progresso guarda cada aula concluída e a melhor nota de cada quiz.

Banco de dados: regras também no PostgreSQL

10
tabelas

15
migrações versionadas

1
função transacional para o conteúdo

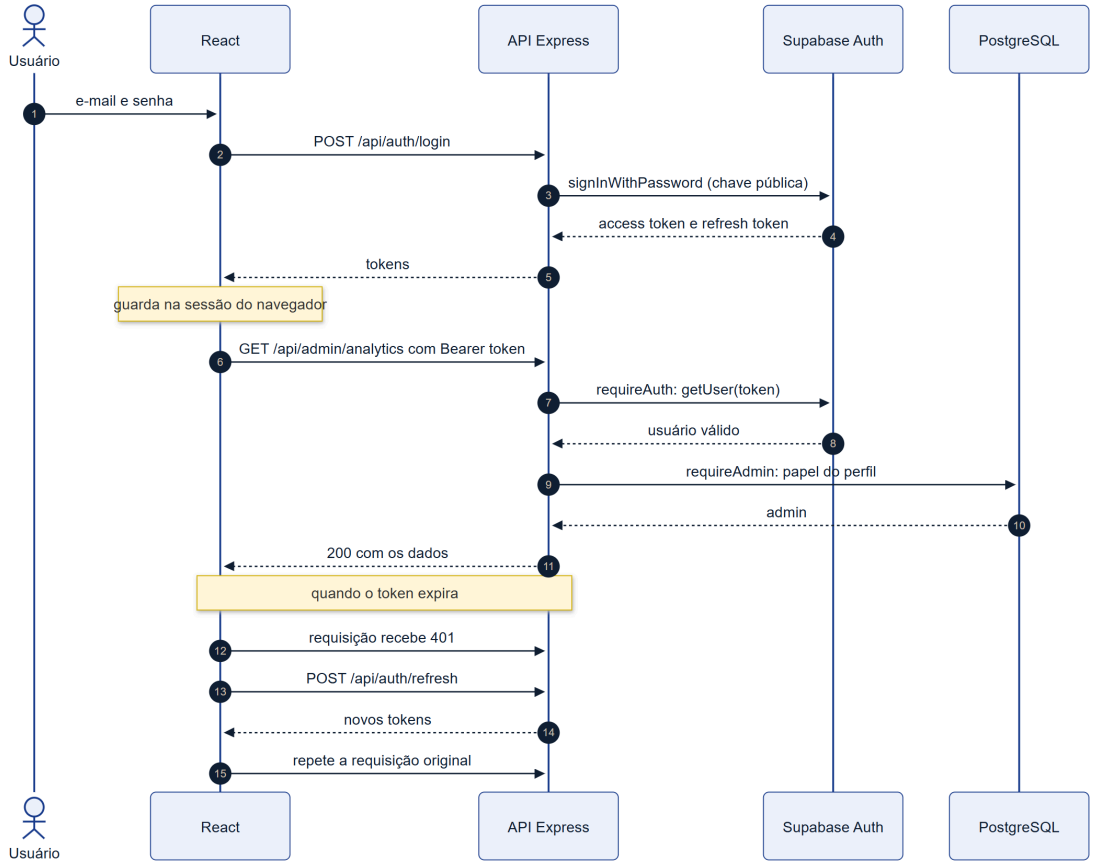
Restrições que protegem os dados

- chaves estrangeiras com exclusão em cascata;
- unique: uma matrícula por aluno e curso;
- unique: uma posição por aula no módulo;
- check: nota entre 0 e 100, status do chamado;
- gatilho cria o perfil quando a conta é criada.

Funções e permissões

- save_course_content salva módulos, aulas e quizzes numa transação;
- aulas existentes são atualizadas, não recriadas, e o progresso fica;
- execução das funções revogada para o acesso público;
- migrações podem rodar mais de uma vez sem erro.

Autenticação e autorização



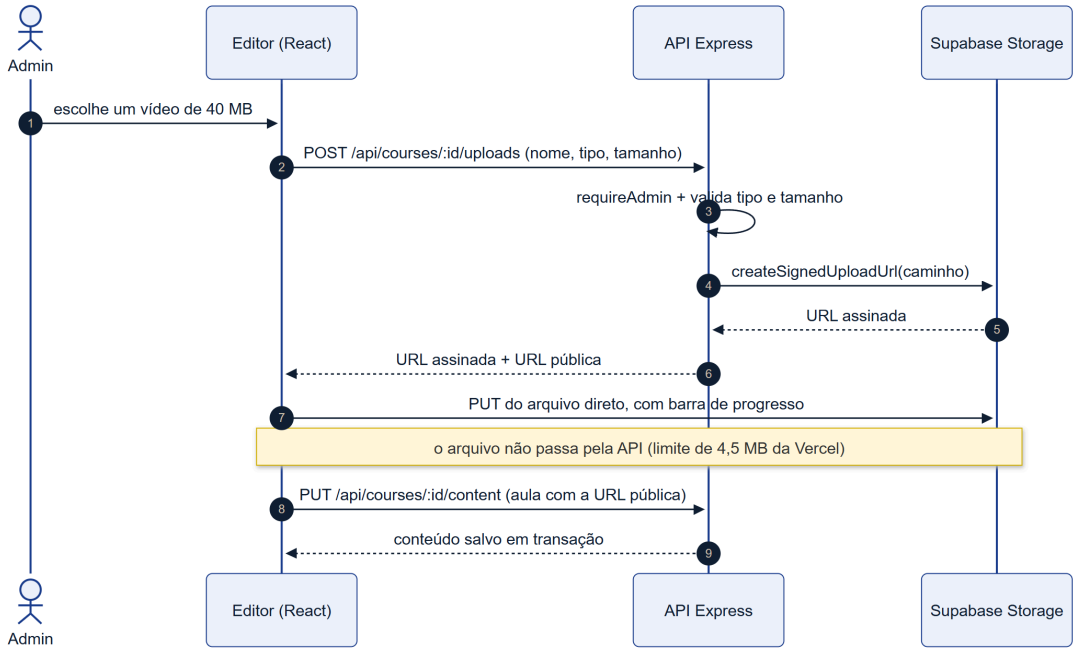
Autenticação: quem é você?

- login pelo Supabase Auth, com senha guardada só como hash;
- a API devolve um token de acesso (JWT) e um de renovação;
- todo pedido leva o token no cabeçalho Authorization;
- quando o token expira, a sessão é renovada sozinha.

Autorização: o que você pode fazer?

- requireAuth valida o token; requireAdmin confere o papel;
- vídeo só para matrícula dentro do prazo;
- visitante vê só a grade; gabarito só para o admin.

Arquivos: upload direto com URL assinada



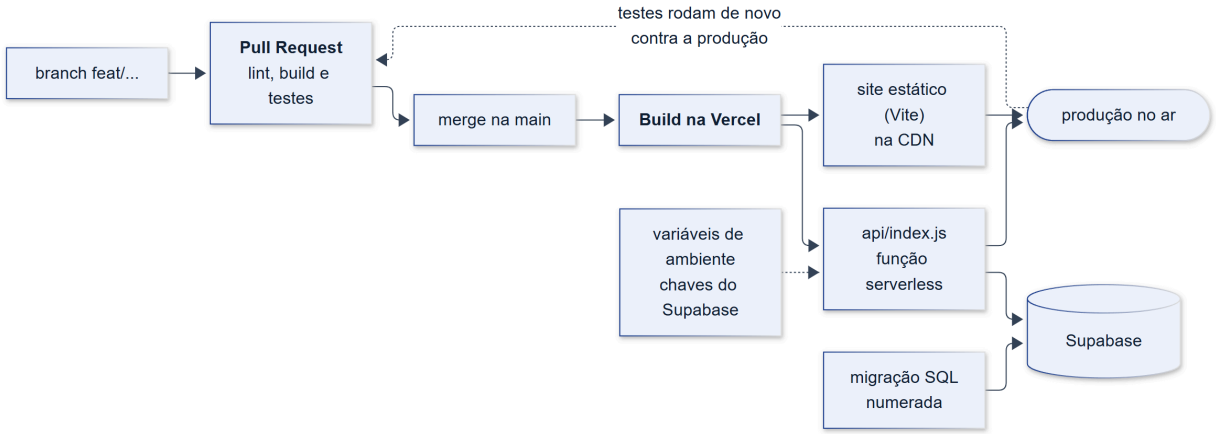
Como um vídeo chega ao Storage

1. o editor pede autorização à API;
2. a API confere o admin, o tipo e o tamanho;
3. o Storage gera uma URL assinada;
4. o navegador envia o arquivo direto, com progresso;
5. a aula é salva com a URL pública.

Por que assim

A Vercel limita cada requisição a 4,5 MB. O vídeo de até 50 MB nunca passa pela API, que só autoriza o envio.

Implantação: do Pull Request à produção



Pipeline

- cada mudança passa por Pull Request com lint, build e testes;
- o merge na main dispara o build na Vercel;
- o site vai para a CDN e a API vira função serverless;
- chaves ficam em variáveis de ambiente, fora do código;
- migrações SQL são aplicadas no Supabase antes do merge.

Resultado

Publicar é uma consequência do fluxo de trabalho: não existe passo manual de deploy. Depois de cada merge, os testes rodam de novo contra a produção.

Regras de negócio principais

Progressão

O próximo módulo só abre quando o anterior está concluído, com o quiz aprovado.

Quiz

Corrigido no servidor. Nota mínima de 70%, e vale a melhor tentativa.

Prazo de acesso

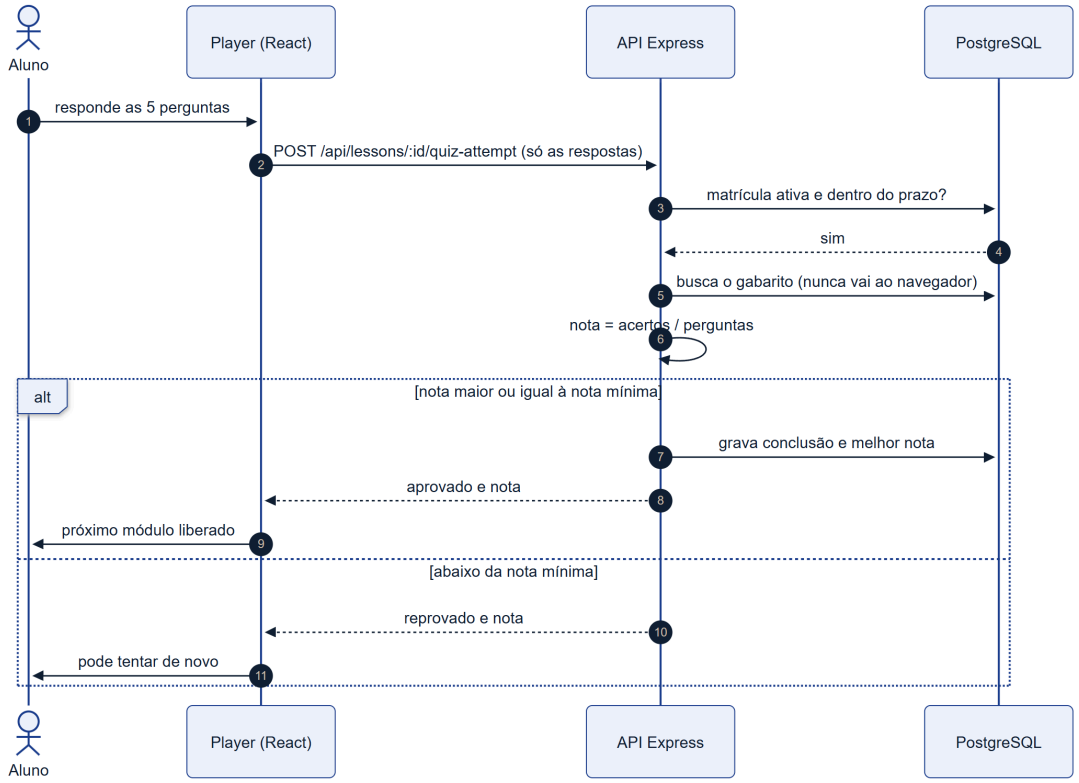
6 meses por matrícula. Expirou: só a grade. Renovar mantém o progresso.

Continue de onde parou

Volta à última aula aberta, ou à próxima pendente, respeitando módulos bloqueados.

Certificado	Regra
Aulas concluídas	100%
Nota final (média da melhor nota de cada quiz)	≥ 70%
PDF	nome, carga horária, nota, data e código

Fluxo crítico: correção do quiz



Passos principais

1. aluno envia só as respostas;
2. API confere matrícula e prazo;
3. gabarito é lido no banco, sem sair do servidor;
4. nota calculada no servidor;
5. aprovado: grava conclusão e melhor nota;
6. player libera o próximo módulo.

Por que no servidor?

Se a correção fosse na tela, bastaria abrir as ferramentas do navegador para ver o gabarito ou forjar a aprovação.

Desafios de integração

Desafio	Como resolvi
A Vercel limita cada requisição a 4,5 MB, e os vídeos têm até 50 MB	a API gera uma URL assinada e o navegador envia direto ao Storage
O cliente do Supabase não oferece transação, e salvar um curso mexe em 4 tabelas	uma função no PostgreSQL salva tudo de forma atômica
Reordenar aulas violava a unicidade da ordem	mover a ordem para uma faixa livre antes de reordenar
Express é um servidor; a Vercel roda funções	o app é exportado e reaproveitado como função serverless
A sessão expira no meio do uso	renovação automática com refresh token e repetição do pedido
Links do e-mail de recuperação de senha	URLs de redirecionamento configuradas no Supabase

Verificação, validação e testes

8
suítes de integração

158
verificações

2
ambientes: local e produção

O que os testes garantem

- visitante não recebe vídeo nem gabarito;
- aluno sem matrícula recebe 403;
- quiz não pode ser marcado sem ser feito;
- nota 50% não libera certificado;
- dado inválido não altera o banco;
- acesso expirado bloqueia o conteúdo.

Como são executados

- chamam a API de verdade, com banco real;
- criam usuários e cursos temporários;
- apagam tudo ao final;
- rodam antes de cada Pull Request;
- rodam de novo contra a produção depois do merge.

Verificação e validação

Verificação: ESLint, build e testes conferem se construí certo. Validação: o sistema publicado e usado de verdade confere se construí a coisa certa.

Manutenção, documentação, padrões e reuso

Manutenção (exemplos reais)

- corretiva: retomada de aula devolvia o erro errado;
- evolutiva: prazo de acesso e cursos reais;
- adaptativa: Express adaptado para serverless;
- preventiva: testes de regressão a cada mudança.

Padrões e reuso

- camadas como organização arquitetural;
- middlewares em cadeia (Chain of Responsibility);
- Adapter: um player para YouTube, Vimeo e arquivo;
- clientes do Supabase únicos por módulo;
- reuso: React, Express, Supabase e jsPDF.

Documentação

- README e guia de implantação;
- referência das 36 rotas da API;
- documento de requisitos e este relatório;
- diagramas e migrações comentadas.

Ganho

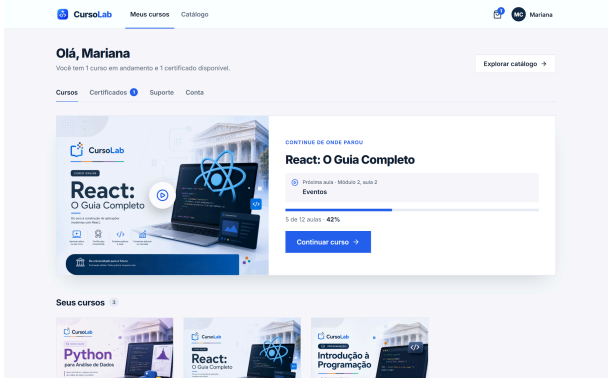
Menos infraestrutura feita do zero, mais tempo nas regras do domínio.

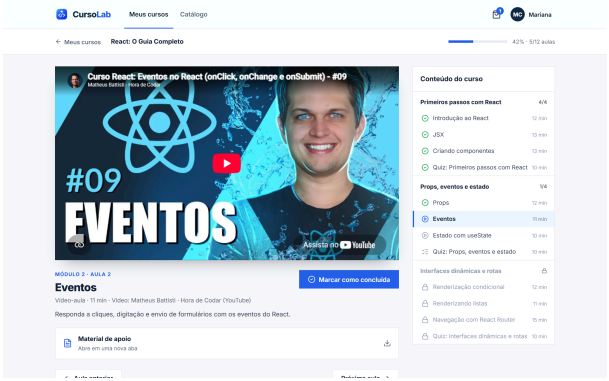
Demonstração

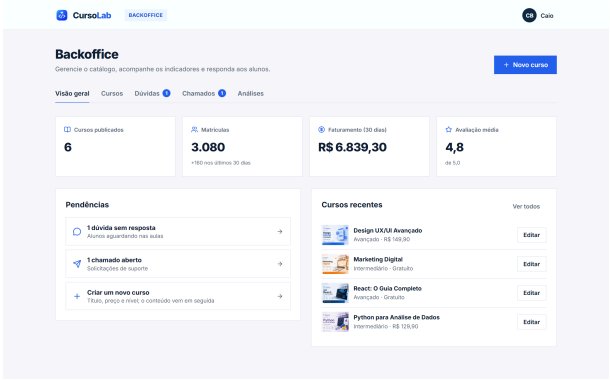
No ar: plataforma-cursos-omega.vercel.app

Roteiro da demonstração

1. landing page → login como aluno
2. catálogo → detalhes do curso → checkout
3. painel: continue de onde parou
4. player: vídeo, apostila e quiz
5. certificado em PDF
6. login como admin: backoffice, editor e chamados







Limitações e trabalhos futuros

Limitações atuais

- pagamento simulado, sem gateway real;
- avaliação do curso pelo aluno não implementada;
- sem notificação por e-mail quando a dúvida é respondida;
- um único papel de administrador;
- parte dos indicadores mistura dados de demonstração.

Evoluções possíveis

- gateway de pagamento (Pix e cartão);
- avaliações e comentários dos cursos;
- notificações por e-mail;
- vários instrutores, cada um com seus cursos;
- testes rodando automaticamente em cada Pull Request (CI).

Conclusão

Resultado

O CursoLab transforma o ciclo de um curso online numa solução web publicada, com requisitos, arquitetura, regras de negócio no servidor, segurança, testes automatizados e documentação.

Conceitos aplicados

- engenharia de requisitos;
- análise e projeto;
- modelagem de dados;
- verificação, validação e testes;
- manutenção e documentação;
- padrões e reuso.

Aprendizado principal

- software não é só código;
- regra importante fica no servidor;
- requisito muda, e a mudança precisa ser rastreável;
- integrar serviços exige mais do que escrever funções;
- teste automatizado dá coragem para mudar.

Obrigado!

Perguntas?

plataforma-cursos-omega.vercel.app